

Patterns in the Chaos

Recognizing what repeats

Pulse Works studio, late evening. Ren sits at his desk surrounded by printed retrospective notes, the Kanban board photo on his monitor, and three weeks of sprint data. Tetsu is refactoring code at the next desk. The office is otherwise empty.

Ren (蓮) "Tetsu, look at this. The Friday crash wasn't a one-off. Three out of our last four incidents happened right after someone pushed a change to an environment Tetsu owned. Same **process flow** (the sequence of steps work follows through a system), same failure point."

→ Ren is beginning to practice *pattern recognition* — identifying recurring themes across separate incidents rather than treating each as isolated.

Tetsu (哲) "Which environment? Dev, staging, or demo?"

Ren (蓮) "All three. That's the point. Every time we push without review, and every time the only person who can fix the fallout is you. It's a **bottleneck indicator** (a sign that a resource or process is constraining **throughput**)."

→ Ren connects the individual incidents to a structural problem — the system itself creates the failure, not any single mistake. This is recognizing *systemic risk*.

Tetsu (哲) "I wouldn't call it a bottleneck. I'd call it poor change management."

Ren (蓮) "It's both. The missing review process is the trigger. But the reason every incident becomes a crisis is that the **WIP limit** on your lane is basically infinity. You're the entire **process flow** for three environments."

→ Ren distinguishes between the immediate cause (no review process) and the structural vulnerability (Tetsu as a single point of failure with no WIP constraints).

Tetsu (哲) "So what's your plan? I can't clone myself."

Ren (蓮) "No, but I can stop treating each crash like bad luck and start treating the pattern as a **systemic risk** (a risk arising from the way the system is structured rather than individual events). The **retrospective** gave us the data. Now we actually use it."

→ This is the shift from reactive firefighting to proactive *pattern recognition* — using retrospective data to identify systemic problems before they recur.

Saki (咲希) *[via text message]* "Saw your message about the incident patterns. Have you mapped the **requirements** for each environment? The overlap might explain why changes cascade."

Ren (蓮) [texting back] "Good call. I hadn't thought of that. You always see the structural angle."

The conversation drifts — environment dependencies, then the retrospective, then the book Saki lent him, then what she's reading now. Ren doesn't notice the time until his phone shows 1:07 AM.

Pattern Recognition

DEFINITION The ability to identify recurring themes or issues in project data.

COMPONENTS Data collection across incidents; comparison of root causes; trend identification; actionable insights.

RELATED root cause, retrospective, lessons learned

EXAMPLE Ren noticed that three of four incidents shared the same process flow and failure point — turning isolated events into a recognizable pattern.

Process Flow

DEFINITION The sequence of steps work follows through a system.

COMPONENTS Inputs and triggers; sequential or parallel steps; decision points; outputs and handoffs.

RELATED value stream, throughput, cycle time

EXAMPLE The XR environment update process flow — code change, push, environment update, verification — had no review step, creating a gap where errors passed through unchecked.

Bottleneck Indicator

DEFINITION A sign that a resource or process is constraining throughput.

COMPONENTS Queue buildup before the resource; idle time downstream; recurring delays at the same point; single-person dependencies.

RELATED throughput, WIP limit, pull system

EXAMPLE Every incident requiring Tetsu's intervention — while three team members sat idle — is a textbook bottleneck indicator pointing to a systemic constraint.

Systemic Risk

DEFINITION A risk arising from the way the system is structured rather than individual events.

COMPONENTS Structural dependencies; recurring failure patterns; cannot be resolved by fixing one instance; requires system-level redesign.

RELATED root cause, process improvement, bottleneck indicator

EXAMPLE Tetsu being the sole person able to fix environment crashes is a systemic risk — even perfect individual performance won't prevent future crises if the structure stays the same.

Review Corner

1 The Queue That Nobody Counted

Ren (蓮) "We set a **WIP limit** of three on the 'In Progress' column, but nobody limited how many items could pile up in 'Waiting for Review.' Tetsu had eleven items in that queue."

→ *WIP limits* (Unit 15): A WIP limit on one stage is meaningless if adjacent stages have no constraints — work simply accumulates at the uncontrolled boundary.

2 The Requirement That Came Back

Saki (咲希) "A user mentioned needing offline mode during the original **requirements** gathering. It was deprioritized. Now three departments are asking for it independently."

→ *Requirements gathering* (Unit 7): Deprioritized requirements don't disappear — they resurface. Tracking them in a backlog prevents the team from re-discovering the same need multiple times.

3 The Throughput Surprise

Tetsu (哲) "We completed more work items this week than last, but the **throughput** that actually reached the client was lower. Most of what we finished was rework."

→ *WIP limits* (Unit 15/16): High activity doesn't equal high throughput. Rework inflates completion counts without delivering new value — a sign the system needs upstream quality improvements.